

Designing a State-of-the-Art Autonomous AI-First iPhone Device Lab for Testing, Debugging, Security Research, and CI at Scale

TL;DR

- **Build a composable control plane, not a monolith.** No single project does this; the winning architecture wraps mature primitives — Appium/WebDriverAgent (UI), pymobiledevice3 + go-ios (device management/diagnostics/tunnels), devicectl (Apple-native lifecycle), Frida (instrumentation) — behind one internal, model-agnostic MCP/REST control plane exposed to Claude Code and OpenCode, with LiteLLM for model routing.
- **Capability depends entirely on device class.** Stock non-jailbroken iPhones give you UI automation, app lifecycle, logs, crashes, screenshots, and per-app-container file access — but NO shell, NO root FS, NO process/memory inspection, and Frida only via gadget-repackaged apps. Deep capabilities (root shell, arbitrary process attach, kernel) require a jailbroken device (checkm8/A11-and-older only for the durable case), Apple's Security Research Device, or Corellium virtualization — each with hard constraints.
- **Scale horizontally around the USB layer.** Plan ~10–15 physical iPhones per USB 2.0 controller/host (endpoint-limited), use industrial Thunderbolt hubs to go higher, prefer detached/prebuilt WDA, and treat jailbroken devices as hostile infrastructure on an isolated network segment. An "ARTEMIS for iPhone" is achievable for UI+diagnostics, but stock iOS cannot reproduce Android's shell/root/Frida-server baseline.

Key Findings

1. **iOS 17+ fundamentally changed device communication.** Apple moved developer services (instruments, debugserver, XCUITest infra) to CoreDevice/RemoteXPC, requiring an encrypted RSD tunnel over IPv6. Direct lockdownd (`StartService`) no longer works for these on iOS 17+. Both pymobiledevice3 and go-ios implement this tunnel; Apple's own (`xcrun devicectl`) handles it natively. This is the single most important architectural fact for any 2025–2026 iOS lab.
2. **UI control ≠ device control ≠ diagnostics ≠ instrumentation.** These are four separate technology stacks: UI control = XCUITest via WebDriverAgent (the ONLY sanctioned way to inject touch/HID on a physical device); device management/lifecycle = devicectl, pymobiledevice3, go-ios, libimobiledevice; diagnostics (logs/crashes/sysdiagnose) = pymobiledevice3/libimobiledevice (stock-capable); instrumentation (Frida/LLDB/memory) = needs jailbreak, gadget-repackaging, or SRD.
3. **Modern iPhones largely cannot be jailbroken durably.** checkm8/palera1n covers only A8–A11 (iPhone X/8 and older). Dopamine covers up to A17/M2 but only on iOS 15–17.x (no public PPL/SPTM bypass for the newest firmware until the reported Dopamine 3.0 in 2026). Your "deep research" fleet must be built from older devices, SRDs, or virtualized Corellium — you cannot assume a current iPhone on current iOS is jailbreakable.
4. **A rich ecosystem of AI/MCP mobile-control projects exists, but all are thin wrappers over the same primitives.** mobile-mcp (mobile-next), iOS-agent-bridge, blitz iphone-mcp, mobile-use (minitap), Maestro MCP, appium-device-farm — none provide multi-tenant scheduling, low-level instrumentation, jailbreak-aware capability negotiation, or fleet management. They are references/seeds, not a finished platform.
5. **The scaling wall is USB, not the Mac.** Per Cambrionix: *"the iPhone 15 only supports USB 2 and the maximum number of endpoints is 8, giving a total number of 15 devices connected to the SuperSync15 before reaching USB's endpoint limit."* Real-world cfgutil ceilings are reported ~22. The legacy OpenSTF "1 iOS device per Mac" limit was an AVFoundation-USB screen-capture constraint, not a macOS limit; WDA's MJPEG server breaks it but still shares 480 Mbps per controller.

Details

1. Current iOS automation landscape (2025–2026)

The iOS automation stack is defined by Apple's iOS 17 CoreDevice transition. Per pymobiledevice3 docs: *"Starting with iOS 17.0, Apple moved developer service access to CoreDevice/RemoteXPC flows, so developer commands (and a few others) need an RSD tunnel to the device."*

([GitHub](#)) DeepWiki's analysis confirms: *"Services like com.apple.instruments.server, com.apple.debugserver, and XCUITest infrastructure no longer respond to direct lockdownd StartService requests on iOS 17+ devices... RemoteXPC services expect IPv6 network communication."*

([DeepWiki](#))

The tunnel uses secure pairing (SRP-3072 with a 6-digit PIN for remote pairing, X25519 key exchange, Ed25519 identity, ChaCha20-Poly1305) creating a QUIC or TCP tunnel over a virtual TUN/TAP interface. ([DeepWiki](#)) pymobiledevice3 can bring this up in-process with no root in most cases; a privileged (`tunneId`) is needed for external tools (lldb), shared/persistent tunnels, or iOS 17.0–17.3.1 on Linux/Windows. ([Doronz88](#))

Practical stacks: **Appium XCUITest driver** (Apache-2.0, actively maintained — v12.x): mature W3C-WebDriver UI layer; client → Appium server → XCUITest driver → HTTP-proxied to WebDriverAgent → XCTest → device. Per Appium docs, *"host-device communication uses usbmux-based transport, Remote XPC tunneling, and xcrun devicectl depending on iOS/tvOS version."* ([Appium](#)) **Apple-native:** (`xcrun devicectl`) (Xcode 15+), (`simctl`), Instruments, LLDB. **pymobiledevice3** (GPL-3.0, pure Python): most complete OSS device-management/diagnostics library. **go-ios** (danielpaulus, MIT): cross-platform Go, JSON output, REST API, ([github](#)) used by HeadSpin and Sauce Labs per its docs. ([Go Packages](#)) **Maestro**: YAML flows, rebuilt its iOS driver around XCUITest.

2. Apple security/automation constraints

Hard limits no tooling circumvents on stock: no arbitrary code execution / no shell; apps sandboxed and signed. **Code signing + provisioning is mandatory** for physical-device installs. Free Apple IDs get 7-day provisioning profiles; paid accounts get 1-year but still require dev certs + UDID registration. ([Capawesome](#)) Apple offers offline provisioning profiles valid 7 days, and extended offline validity by request ([Apple Developer](#)) —

relevant for air-gapped labs. **Developer Mode** must be manually enabled on iOS 16+ (toggle + reboot), hard to fully automate. **The only sanctioned touch-injection path is XCTest/XCUITest** running as an on-device bundle (WDA). Per iOS-agent-bridge: *"iOS allows only one sanctioned way to inject hardware and touch events into a physical device: XCTest / XCUITest, running as an instrumentation bundle on the phone."* (GitHub)

3. Stock (non-jailbroken) device capabilities

CAN do (verified): discover, pair, connect USB+network, install/uninstall IPAs (devicectl, ideviceinstaller, go-ios), launch/terminate apps (devicectl `process launch --console` captures stdout as of Xcode 16), reboot; full UI automation via WDA (tap/swipe/type/long-press, read accessibility hierarchy, screenshot, MJPEG stream, SpringBoard/Settings/permission dialogs, Safari, arbitrary apps); query installed apps; access the app's OWN container files via AFC/house-arrest for dev-signed apps; collect syslog/unified logs (idevicesyslog, pymobiledevice3), crash reports (CrashReportsManager), sysdiagnose, diagnostics, battery, disk; packet capture (pymobiledevice3 PCAP), install HTTP proxy profiles (go-ios); performance via DVT/Instruments over the tunnel (process list, sysmontap CPU/memory, energy); LLDB attach to development-signed/debuggable apps only; Frida ONLY via gadget repackaging (§11).

CANNOT (stock): get a shell, access broader/root filesystem, list/kill arbitrary system processes, dump arbitrary process memory, attach to system daemons or App Store apps, inspect keychain broadly, load system-wide instrumentation, or run frida-server.

4. Jailbroken device capabilities and the jailbreak matrix

Jailbreak unlocks: root shell over SSH, AFC2 (full filesystem), frida-server (attach to any process, memory read/write, hook system frameworks), arbitrary process inspection, keychain dumping (authorized envs), system daemon interaction.

Jailbreak reality (Nullcon Goa 2025 / iClarified / cfw.guide / palera1n team):

Tool	Devices	iOS	Type	Automation suitability
palera1n (checkm8)	A8-A11 only (≤ iPhone X/8)	15.0–latest on those SoCs	Semi-tethered, rootless/rootful; unpatchable bootrom	Excellent for dedicated research (unpatchable); semi-tethered = re-tether on reboot; A11 requires passcode disabled
Dopamine (opa334)	A8-A17/M1-M2	15.0–17.x; iOS 26 A12-A13 per 2026 update	Semi-untethered (reopen app after reboot), rootless	Best modern coverage; re-sign every 7 days on 16.7+; reboot requires re-jailbreak tap
TrollStore	arm64 (A8-A11) via palera1n; arm64e (A12+) 14.0–16.6.1, 17.0, 16.7 RC	—	NOT a jailbreak — permanent arbitrary-entitlement sideloading	Very useful: install unsigned/entitled binaries permanently without 7-day expiry
checkm8/checkra1n	A5-A11 (bootrom)	—	Hardware exploit	Foundation of palera1n

Critical: checkm8 applies only to A5-A11. Modern iPhones (A12+ / arm64e) on current iOS have **no public durable jailbreak**. idownloadblog/Nullcon 2025 confirms the full modern-device jailbreak "still requires a PPL/SPTM bypass, which is not yet publicly available." (GitHub) Dopamine 3.0 (2026) reportedly adds Titan PPL/SPTM + momentarius bypasses expanding coverage, (iClarified) but treat as evolving; never assume a given current device/iOS is jailbreakable.

Best dedicated research devices: iPhone 8 / iPhone X (A11) or earlier on iOS 15–16 via palera1n rootless (unpatchable, stable), plus A12-A13 on iOS 15–16 via Dopamine. Keep these as fixed, never-updated, never-daily-use lab assets.

5. Apple Security Research Device (SRD)

Apple's sanctioned answer to "a jailbroken modern iPhone." Per security.apple.com: *"The Security Research Device (SRD) is a specially fused iPhone that allows you to perform iOS security research without having to bypass its security features. Shell access is available, and you can run any tools, choose your own entitlements, and even customize the kernel. All iOS and iPhone components are eligible for SRD Program research, except Apple Pay and third-party apps."* (Apple) Researchers can *"Install and boot custom kernel caches. Run arbitrary code with any entitlements, including as platform and as root outside the sandbox. Set NVRAM variables. Install and boot custom firmware for SPTM and TXM."* (AppleInsider) The 2026 program *"now includes iPhone 17 devices with...Memory Integrity Enforcement"* (Apple) (security.apple.com/blog), and discovered vulnerabilities feed Apple Security Bounty, whose *"highest award will be \$2 million before bonus considerations"* effective November 2025. (Apple)

Constraints making SRD a poor general-lab fit: 12-month renewable loan, remains Apple's property, **must remain on program-participant premises**, access limited to Apple-authorized people, discovered vulns must be reported to Apple, (Apple) limited quantity. On application dates: press coverage (AppleInsider, SecurityWeek, 9to5Mac, Sept 2025) states the 2026 SRDP *"application period ends October 31, 2025,"* (SecurityWeek) while Apple's own page reads *"This year's application period ends October 30, 2026"* (Apple) (security.apple.com/research-device) — confirm the current window directly with Apple. Modern-silicon deep access, but legal/operational strings incompatible with a freely-orchestrated autonomous fleet.

6. Corellium (virtualized iOS)

ARM-native virtualized iPhones via CHARM type-1 hypervisor (Corellium) (AWS Graviton or on-prem/air-gapped appliances). Instant root/jailbreak without exploits, kernel debugging, filesystem access, syscall analysis, network inspection, snapshot/clone, (Corellium) REST API.

Won a fair-use ruling vs Apple (2020, upheld 2023). ([Developer Tech News](#)) Cellebrite acquired Corellium for an enterprise value of \$170M (announced June 5, 2025; completed December 2, 2025): "\$150 million was paid in cash at closing, with \$20 million converted to equity," plus "up to an additional \$30 million in cash based on the achievement of certain performance milestones over the next two years" (Cellebrite/GlobeNewswire). On pricing, Corellium cloud individual plans start at \$99/month (2-core CPU), but "users planning to virtualize iOS running on newer Apple devices will need the 6-core plan, which costs \$295 per month" ([9to5Mac](#)) (9to5Mac, Jan 2021); its AWS Graviton-based Arm Virtual Hardware bills "at a rate of \$0.50 per core hour" ([Corellium](#)) (support.avh.corellium.com).

For an on-prem closed org, Corellium's air-gappable appliance is the single most powerful option for deep research on MODERN iOS versions that cannot be physically jailbroken — it complements (does not replace) a physical fleet for hardware/sensor/radio fidelity. Commercial/closed-source, significant budget line.

7. Simulator capabilities

The iOS Simulator ([simctl](#), Xcode) runs on macOS — excellent for deterministic UI/functional CI at high density and low cost. Supports install/launch, UI automation (XCUITest, Maestro via IDB, Appium), screenshots, logs, biometrics simulation. But per HackTricks: "A simulator is not the same as an emulator. It runs a platform model... does not reproduce every hardware-backed security property of a physical device." ([HackTricks](#)) No real Secure Enclave, no radios/sensors, low security-research fidelity. Use for scale-out deterministic functional tests, not security/hardware-dependent behavior.

8. XCUITest / WebDriverAgent / Appium analysis

WDA is an XCTest-based HTTP server installed on the device (WebDriverAgentRunner-Runner); Appium's XCUITest driver manages its lifecycle and proxies W3C commands. Key facts:

- **Signing/provisioning is the perennial pain** — WDA must be code-signed for real devices; automate with a stable dev cert + registered UDIDs, or install prebuilt WDA via go-ios ([ios runwda](#)) or ([useXctestrunFile](#)).
- **Parallel sessions** require unique ([wdaLocalPort](#)) (default 8100), unique ([mjpegServerPort](#)) (default 9100), unique ([derivedDataPath](#)) per device. Sharing 9100 causes video cross-talk (appium #14668/#16556).
- **Detached WDA** (go-ios/iproxy rather than Appium spawning xcodebuild) improves performance and removes persistent xcodebuild cost.
- **Footprint**: xcodebuild ~120MB RAM per live instance, up to 15s build overhead even with cache (Thuyen Trinh 2025); on-device WDA memory-leak growth ~20MB→~2GB then crash over long sessions (appium #15457), driving a "restart WDA every few hours" rule.
- **iOS 26 caveat**: go-ios #631 (Oct 2025) shows ([runwda](#)) tunnel failures on iOS 26 still being worked — the stack needs continuous maintenance against new iOS.

Production scale evidence: No vendor publishes a physical-iPhones-per-Mac ratio. AWS Device Farm documents a conservative model: "Amazon-managed macOS instances (hosts) that dynamically connect to the iOS device during the test run" ([AWS](#)) — effectively one host per device during a run. Agoda's cited "200 Mac minis" farm is **simulators on virtualized macOS**, NOT physical iPhones (64GB RAM minis, QEMU/KVM virtualizing macOS) ([Medium](#)) — do not use as physical-device evidence. Community reports show 3+ physical devices per Mac mini via Appium with distinct ports. The binding constraint is USB (§29).

9. Apple-native tooling deep dive

- **devicectl** (Xcode 15+): ([device install app](#)), ([device process launch --console --terminate-existing](#)), ([HackTricks](#)) ([device info](#)), ([list devices](#)), reboot; establishes CoreDevice tunnels natively; Node wrapper appium/node-devicectl (Apache-2.0). Limitations: no documented direct debugserver launch; some operations "not implemented on this device."
- **simctl**: full simulator control. **Instruments/DVT**: perf/energy/allocations, programmatically via pymobiledevice3 over the tunnel.
- **OSLog/Unified Logging, MetricKit, crash reports, sysdiagnose**: collectable; unified logs volatile (hours→~30 days, ElcomSoft), ([ElcomSoft](#)) collect promptly.
- **Xcode Cloud**: cloud-only, unsuitable for closed on-prem.
- **MDM/supervision**: Apple Configurator/[cfgutil](#) for supervision/install/restrictions; ([cfgutil](#)) reportedly stalls ~22 devices (Apple forum).

Automatable headlessly EXCEPT initial Developer Mode enablement, first-time pairing trust (mitigate via supervision or go-ios pairing), and signing cert provisioning.

10. libimobiledevice / pymobiledevice3 ecosystem — the ADB-like abstraction

pymobiledevice3 (GPL-3.0) is the strongest ADB analog. README: "device discovery, port forwarding, syslog/oslog streaming, app & profile management, AFC file access, crash reports, PCAP sniffing, firmware update, recovery/DFU, backup/restore, WebInspector automation, a CDP bridge... and DDI/DVT developer tooling (iOS 17+ over a tunnel)." ([GitHub](#)) Ships a Claude Code plugin ("device-operator agent skill"). Its ([tunnel](#)) FastAPI server auto-discovers devices via USB (RSD), WiFi (RemotePairing), usbmux, mobdev2. ([DeepWiki](#))

go-ios ([MIT](#)) complements it: production CLI + REST API, ([tunnel start](#)), ([GitHub](#)) ([runwda](#)), ([image auto](#)), app install/launch, crash/diagnostics, ([devicestate](#)) (thermal/network emulation), ([Go Packages](#)) MJPEG/h264 stream, JSON output.

libimobiledevice ([LGPL](#)) — the C foundation (usbmuxd, ideviceinstaller, idevicesyslog, idevicecrashreport, iproxy, ifuse); still relevant but surpassed by pmd3/go-ios for iOS 17+ tunnels.

Verdict: pymobiledevice3 + go-ios together ARE the "internal ADB for iOS." Neither requires jailbreak for stock-device features.

11. Frida / runtime instrumentation

Two iOS modes (frida.re): **Jailbroken: frida-server as root** — full power (attach any process, hook system frameworks, memory r/w), requires root (ptrace), (Koz) jailbreak only. **Non-jailbroken: Frida Gadget in a repackaged IPA** — inject (FridaGadget.dylib) (objection (patchipa), insert_dylib/optool), re-sign with dev cert, install; instruments ONLY that app, and you must be able to re-sign (org apps: fine; App Store apps: need a decrypted IPA from a jailbroken device first).

Critical jailed constraint (frida.re): "on a jailed iOS device the only way to use the Interceptor API is if a debugger is attached prior to Gadget being loaded... the relaxed code-signing state is sticky once set." (Frida) Community reports (frida #3621) confirm gadget-patched apps crashing on launch on iOS 17+ jailed devices — expect fragility. **Objection** builds on Frida; **LLDB** (via debugserver over the tunnel) is the debugger path for dev-signed apps.

MCP capability exposure must be dynamic: (ios.trace.function), (ios.trace.objc), (ios.memory.read), (ios.process.attach), (ios.shell.exec) advertised ONLY for devices whose capability profile supports them (jailbroken/SRD/Corellium). On stock, expose only (ios.instrument.attach) scoped to gadget-repackaged org apps.

12. Existing AI mobile-agent projects

Project	iOS physical	Multi-device	AI-native/MCP	Low-level	Verdict
mobile-mcp (mobile-next)	Yes (WDA)	Limited	MCP, accessibility-first (no VLM needed)	No	Best reference/seed for UI MCP; token-efficient a11y snapshots
iOS-agent-bridge (UgeeCodes)	Yes, iOS 17+	Single	MCP + WDA over CoreDevice IPv6 tunnel; a11y-tree refs not raw coords	No	Strong iOS-17-tunnel + WDA + MCP reference
blitz iphone-mcp (blitzdotdev)	Yes + sim	Single	MCP; auto-configures Claude Code, Cursor, Codex, OpenCode	No	Good harness-integration reference
mobile-use (minitap)	Android + iOS	No	LLM-agnostic, OpenAI-compatible base URL	No	Agent-loop + LiteLLM-compat reference
droidrun/mobilerun	iOS via portal	No	LLM-agnostic, a11y+screenshots	No	Android-first; agent-loop reference
arbigent (takahirom)	iOS/Android/Web	Sharding	AI scenarios, result caching	No	Deterministic+AI blend + caching idea
Lakr233/iphone-mcp	Yes (Appium)	No	MCP over HTTP	No	Appium-MCP reference
witchan/ios-mcp	Jailbroken only	No	MCP with (run_command) shell, file r/w	Yes (JB)	Jailbroken-device MCP tool-surface reference
mirroir-mcp (jfarcan)	via iPhone Mirroring	No	MCP screenshot/tap	No	macOS-Mirroring; limited/fragile

None provide scheduling, fleet health, jailbreak-aware capability negotiation, observability, or CI integration. Adopt their tool schemas and a11y-tree approach; build the platform around them.

13. Existing MCP projects — build vs reuse

The mobile MCP ecosystem is UI-centric. mobile-mcp's accessibility-first design ("drives apps from the native accessibility tree (no vision model, no image tokens), falling back to screenshots + coordinates only when needed") (GitHub) is the correct default for cost/latency. **No existing MCP covers device management + diagnostics + instrumentation + scheduling.** Reuse a UI MCP as one module; BUILD the devices/diagnostics/instrumentation/scheduler MCP modules internally.

14. Existing device-farm projects

- **OpenSTF/DeviceFarmer:** Android-focused, effectively unmaintained for modern OS (OpenSTF v3.4.1 = Android 9); (dev) iOS via (stf_ios_support) limited — FAQ: "Why can't I use/provide multiple IOS devices simultaneously from a MacOS machine? A: This is a technical limitation within the video streaming mechanism." (legacy AVFoundation-over-USB path, not a fundamental limit).
- **appium-device-farm** (AppiumTestDistribution): MIT + proprietary components; active; device management + parallel Appium sessions. **Strong adopt candidate** for scheduling/session management.
- **Sonic, Zebrunner MCloud, DeviceLab:** various maturity; mostly Android or commercial.
- **Commercial (BrowserStack/Sauce/AWS/HeadSpin/Bitbar):** architectural lessons only (one-host-per-device model, per-device port isolation, health monitoring).

15. ARTEMIS comparison — what "ARTEMIS for iPhone" requires

Google's ARTEMIS, "built by Google's Pixel-Test-Engineering (PTE) Fusion team," (GitHub) turns natural-language into Android automation, integrates via MCP with Antigravity/Claude Code/Codex, (GitHub) and reports "99%+ task completion on Google Research's AndroidWorld benchmark (100+ multi-step tasks)" (GitHub) — AndroidWorld itself comprises "116 programmatic tasks spanning 20 real-world applications." (arxiv) It relies on ADB, UIAutomator, Logcat, Android shell, scrcpy, Frida — all free on stock Android with USB debugging.

The asymmetry: Android's stock baseline (ADB shell, `pm`/`am`, logcat, root-optional Frida, scrcpy mirroring) does not exist on stock iOS. To build "ARTEMIS for iPhone": UI + app lifecycle + logs + crashes + screenshots + perf is **achievable on stock** (WDA + pmd3/go-ios + devicectl); but shell, arbitrary process list/kill, root FS, system-wide Frida, syscall trace are **NOT achievable on stock iOS** — needing jailbreak (old devices), SRD, or Corellium. So an iPhone ARTEMIS is two-tier: a broad stock tier (UI/diagnostics/CI) and a narrow deep tier (jailbroken/SRD/Corellium). The AI must negotiate capabilities per device.

16. Comprehensive capability matrix (summary)

Legend:  Supported ·  Partial/conditional ·  Unsupported

Capability	Stock/Dev	Supervised/MDM	Jailbroken	SRD	Simulator
Discover/pair/connect USB+net	 pmd3/go-ios/devicectl				 simctl
Install/uninstall IPA	 devicectl	 (+MDM push)			
Launch/terminate/reboot					
UI tap/swipe/type/hierarchy	 WDA				
Screenshot/stream	 WDA MJPEG				
SpringBoard/Settings/permissions	 WDA				
Automate Safari/arbitrary apps					
App's own container files	 dev-signed only		 full FS		
Broader/root filesystem			 AFC2		 host FS
Shell (mobile/root)			 SSH root		 host shell
List/kill processes	 own via DVT				
Dump process memory			 Frida		
Attach LLDB	 debuggable apps		 any		
Syslog/unified/crash/sysdiagnose	 pmd3				
Packet capture	 pmd3 PCAP				
Perf (CPU/mem/energy)	 DVT				
Frida function/objc trace	 gadget-app only		 system-wide		
Hook/modify runtime	 gadget app				
System daemon interaction					
Keychain inspection			 (authorized)		
Entitlements/signing inspection					
Kernel customization			 (rootful/old)		

17. Device connectivity architecture

- **Transport:** USB primary (reliability, power), WiFi/RemotePairing secondary. iOS 17+ requires the RSD tunnel — run `go-ios tunnel start` or `pymobiledevice3 tunnel` per host as a privileged launchd daemon.
- **Pairing:** automate via go-ios (pairs without manual tap in many cases) or supervision; store pairing records securely; handle trust dialog via supervision where possible.
- **Developer Disk Image:** auto-mount via `go-ios image auto` / Xcode.
- **Reconnection:** usbmuxd + tunnel auto-rediscovery; a per-device supervisor re-establishes tunnel + WDA on disconnect.

18. Remote device-control architecture

Layered: (1) Physical — iPhones on industrial managed USB hubs → Mac workers. (2) Per-device agent on the Mac worker — owns the tunnel (go-ios/pmd3), the WDA session (unique ports), diagnostics collectors, and (research devices) the Frida/SSH channel. (3) Worker node service — registers devices, exposes local gRPC/HTTP API, enforces isolation. (4) Control plane — device registry, scheduler, artifact store, observability, MCP/REST gateway. (5) Harness — Claude Code / OpenCode connect to the MCP gateway.

19. Device scheduler

Reservation-based, keyed on a per-device capability document:

```
device:
  udid: 00008030-...
  model: iPhone10,3 # iPhone X
  ios: "16.7.2"
  soc: A11
  class: research # stock | supervised | research | srd | corellium | simulator
  jailbroken: true
  jailbreak: palera1n-rootless
  frida: true
  shell: true
  host: mac-worker-03
  health: healthy # healthy | degraded | quarantined | maintenance
  wda_port: 8102
  mjpeg_port: 9102
```

Features: attribute selection (e.g. `(ios >= 17 AND class == stock)`), leases with TTL + heartbeat, locking against double-booking, priority queues (CI > interactive > exploratory), affinity (same device for reproduction), quarantine on repeated failures, maintenance windows, automatic recovery hooks. appium-device-farm can seed this; extend with capability/jailbreak awareness.

20. AI-agent architecture (goal-driven)

Agents receive GOALS, not scripts. Multi-agent roles: **Explorer** (maps UI, a11y-tree first, screenshots fallback); **Tester** (executes/authors deterministic flows; prefers XCTest/Maestro/Appium over LLM actions for known paths); **Investigator** (on failure collects logs/crashes/traces, forms hypotheses, reproduces across devices/iOS versions, classifies root cause: app / OS-version / device / infra); **Security** (on research devices, drives Frida/LLDB/network capture within authorization scope); **Verifier** (confirms fixes, produces auditable evidence). The agent negotiates device capabilities from the registry before choosing tools, reserves devices, releases on completion.

21. Claude Code integration

Expose the lab as MCP servers. Claude Code reads `(.mcp.json)` / `(~/ .claude.json)`; `(Claude Code Docs)` supports stdio and Streamable HTTP (Anthropic's recommended transport as of 2026; SSE deprecated). `(Build Fast with AI)` Project-scoped `(.mcp.json)`:

```
json
{
  "mcpServers": {
    "ios-devices": { "type": "http", "url": "https://lab.internal/mcp/devices", "headers": { "Authorization": "Bearer ${LAB_TOKEN}" } },
    "ios-ui": { "type": "http", "url": "https://lab.internal/mcp/ui" },
    "ios-diag": { "type": "http", "url": "https://lab.internal/mcp/diagnostics" },
    "ios-instr": { "type": "http", "url": "https://lab.internal/mcp/instrumentation" }
  }
}
```

Use HTTP transport for shared multi-user access (stdio is per-user/local). Treat tool output as untrusted (prompt-injection fencing).

22. OpenCode integration

OpenCode uses a different schema: top-level `(mcp)` key (not `(mcpServers)`), `(command)` as an array, `(environment)` (not `(env)`), `(local)`/`(remote)` types. `(DIY AI)` Config in `(~/ .config/opencode/opencode.json)` (global) or `(opencode.json)` (project). `(DIY AI)` Do NOT paste Claude Code blocks unchanged. Example:

```
json
{
  "mcp": {
    "ios-devices": { "type": "remote", "url": "https://lab.internal/mcp/devices", "enabled": true }
  }
}
```

A single HTTP MCP gateway serves both harnesses identically — the key reason to prefer HTTP transport and a model-agnostic tool surface.

23. LiteLLM / model architecture

LiteLLM proxy (BerriAI, open source, self-hostable, air-gap capable) exposes one OpenAI-compatible endpoint routing to any backend `(LiteLLM)` (vLLM, Ollama, internal models). Capability classes as model aliases: **LOW** (device actions, a11y-tree reasoning, simple taps — small fast VLM/text); **MID** (test authoring, UI exploration, log triage — mid VLM); **HIGH** (crash root-cause, security reasoning, cross-device

hypothesis — strongest model). Auto-routing/aliases let the agent escalate LOW→HIGH without code changes. (LiteLLM) Prefer accessibility trees over screenshots to cut image tokens/latency; screenshots only when a11y insufficient. Route concurrency through LiteLLM load balancing. (LiteLLM)

24. CI pipeline (vendor-neutral)

Git push → Build IPA (on-prem Mac build host; signing via internal cert + registered UDIDs or TrollStore for entitled test builds) → Select device matrix (scheduler query) → Reserve → Deploy (devicectl) → Deterministic tests (XCUITest/Maestro/Appium) → AI exploratory (Explorer/Tester) → Security tests (research devices, Frida/LLDB) → Collect evidence → AI failure investigation → Cross-device comparison → Report → Release. Integrate via GitLab CI/GitHub Actions/Jenkins runners calling the lab's REST API. Keep the interface vendor-neutral (REST + MCP) so the CI system is swappable.

On-prem signing implication: you need a persistent Apple Developer account, a dev/distribution certificate, registered device UDIDs; provisioning profiles expire (7 days free / 1 year paid) (Capawesome) and must be auto-renewed by a signing service. Enterprise in-house distribution (\$299/yr) (Capawesome) simplifies fleet install but Apple has tightened enterprise cert policy. Air-gapped: use offline provisioning profiles (7-day, or extended by Apple request) (Apple Developer) and a local re-signing pipeline.

25. Security-research workflow

Authorized org-owned/approved targets only. On research devices: runtime instrumentation (Frida), crash reproduction, fuzzing harnesses, network inspection (proxy profile + PCAP), sandbox/filesystem inspection, process/memory inspection. Every action capability-gated and logged. Isolate potentially-malicious test apps: run only on quarantined research devices on an isolated VLAN with no route to the control plane; treat the device as compromised after malware tests and re-image (restore) before reuse.

26. Observability

OpenTelemetry-based. Capture per action: task_id, pipeline_id, agent_id, model + capability class, device_udid/model/ios/jailbreak_state, host, WDA_session/ports, action, a11y-tree snapshot, screenshot ref, logs, crashes, traces, Frida events, LLM request/response + tokens + latency, tool calls, errors, retries. Artifacts in on-prem object storage (MinIO). Every autonomous run must be replayable (persist full action + observation sequence) and auditable (immutable log). LiteLLM provides per-key spend/latency tracking; (LiteLLM) export to the same OTel backend (Grafana/Tempo/Loki).

27. Reliability / recovery

Failure	Detection	Automated recovery
USB disconnect	tunneld/usbmux event	replug via managed-hub port toggle (Acroname/Cambrionix API); re-pair; restart tunnel+WDA
Dead/unresponsive device	heartbeat timeout	power-cycle port; if persists → quarantine
Low battery	diagnostics poll	route charging; deprioritize in scheduler
Locked device	WDA status	WDA unlock with stored passcode (test devices only)
Expired provisioning	install/build error	signing service re-provisions + reinstalls WDA
WDA crash / memory leak	health probe on :8100	scheduled restart every N hours; on-demand restart
Appium/Xcode failure	process monitor	restart worker service; rebuild derivedData
Host reboot	node registration loss	launchd auto-start; scheduler drains + re-registers
iOS update drift	version mismatch	pin devices; block OTA via supervision
Jailbreak instability	SSH/frida probe	re-jailbreak (semi-tethered tether); quarantine if unstable
Agent loop / model timeout	step budget + watchdog	abort, capture state, release device, escalate
Malformed UI tree	schema validation	fallback to screenshot+VLM; retry

28. Control-plane security architecture

- **Network segmentation:** three zones — control plane (trusted), stock-device workers (semi-trusted), research/jailbroken workers (untrusted VLAN, no inbound to control plane; outbound via broker only).
- **Jailbroken/malware devices = hostile:** assume compromise; no secrets on them; re-image after security runs.
- **Secrets:** signing certs/keys in a vault (internal HashiCorp Vault); never plaintext on worker disks; short-lived tokens.
- **MCP authorization + RBAC:** per-tool scopes; shell/Frida/memory tools require elevated role + device-class match; deny-by-default.
- **Audit:** immutable logs of every tool call, reservation, artifact access.
- **Sandboxing:** worker services least-privilege; per-device agents isolated (separate users/containers where macOS allows).
- **Artifact retention:** policy-based TTL; PII/scoping controls.

29. Scaling analysis (evidence-based)

The binding constraint is the USB layer, far below the theoretical 127-device USB tree limit. Cambionix documents that the iPhone 15 is USB 2.0 with a maximum of 8 endpoints, yielding "a total number of 15 devices connected to the SuperSync15 before reaching USB's endpoint limit." Endpoint usage varies by model — Cambionix's technical note observes "Each USB device can define up to 32 endpoints...but often uses much less; for example, an iPhone 11 will use nine endpoints"— so real ceilings differ per device. Real-world (cfigut1l) ceilings of ~22 devices are reported on Apple's forums. All USB 2.0 devices behind one controller share 480 Mbps, and concurrent MJPEG streams compete for it.

- **Devices per USB 2.0 controller/hub:** ~10-15 (endpoint-limited); leave headroom → design ~10-12/hub.
- **Break the wall** with Thunderbolt industrial hubs (Cambionix ThunderSync, Acroname USBHub3c/USBHub3+) or multiple independent USB host controllers per Mac. Managed hubs give per-port power control for automated reboot/recovery.
- **WDA/xcodebuild** is the second constraint: ~120MB RAM per live xcodebuild + periodic WDA restarts; use prebuilt/detached WDA.
- **Host CPU/RAM** is rarely the first wall for PHYSICAL devices (compute runs on the phone) — but IS the primary wall for SIMULATOR farms (Agoda: 64GB RAM minis).

Design targets: 10-20 devices → 1-2 Mac workers, 1-2 managed hubs. 50 → 4-6 Mac workers, Thunderbolt hubs, HA control plane, queueing. 100-500+ → many Mac workers in racks, sharded scheduler, centralized artifacts, per-worker isolation, aggressive health automation; strongly consider Corellium for modern-iOS deep-research density (physical fleet stays for hardware fidelity). At 500+: control-plane HA/sharding mandatory, device-health automation fully hands-off, network/artifact (log/video) bandwidth a first-class capacity item.

30. Hardware requirements

- **Mac workers:** Apple Silicon Mac mini (M-series), 32-64GB RAM (higher if also running simulators), fast SSD. No IPMI — plan smart PDUs + USB/IP KVM for out-of-band (VDMs don't pass through hubs; one host per target for DFU). (Acroname)
- **USB:** industrial managed hubs (Cambionix SuperSync15/ThunderSync; Acroname USBHub3c 6×100W (Acroname) or USBHub3+ 8-port) for per-port power control and endpoint headroom.
- **Power/thermal:** per-port 7.5-10W charging; racks with cooling; battery management (keep 30-80% for longevity).
- **Recovery:** managed-hub port toggle for USB reset/power-cycle; DFU via Acroname USBHub3c for re-imaging. (Acroname)
- **Research devices:** fixed older A11/A12-A13 iPhones on pinned iOS; optional SRDs; optional Corellium appliance.

31. Build-vs-adopt analysis

Component	Decision	Rationale
UI automation	Adopt Appium XCUITest + WDA	Only sanctioned touch path; mature, Apache-2.0
Device mgmt/diagnostics/tunnel	Adopt pymobiledevice3 + go-ios	Most complete iOS-17 tunnel + diagnostics; GPL-3.0/MIT
App lifecycle	Adopt devicectl (+ node-devicectl)	Apple-native, supported
Instrumentation	Adopt Frida + Objection; wrap capability-gated	Standard DBI
UI MCP	Fork/compose mobile-mcp / iOS-agent-bridge	Good a11y-first tool schema
Scheduler/registry	Fork+extend appium-device-farm OR build	Needs jailbreak/capability awareness it lacks
Devices/diag/instr MCP	Build internal	No project covers this
Model routing	Adopt LiteLLM	Air-gap OpenAI-compatible gateway
Observability	Adopt OpenTelemetry + MinIO + Grafana stack	Standard
Deep modern-iOS research	Adopt (commercial) Corellium; and/or SRD	Only durable modern-iOS root

Overall: Build an internal control plane that composes these primitives. Do not adopt any single project as the platform.

32. Recommended architecture (final stack)

Layer	Technology	Existing project	Build/Fork/Adopt	Why	License	Maturity	Risk
UI control	WDA via Appium XCUITest	appium-xcuitest-driver	Adopt	Only sanctioned HID path	Apache-2.0	High	iOS churn
Device mgmt/diag	pymobiledevice3 + go-ios	doronz88, danielpaulus	Adopt	iOS-17 tunnels, diagnostics, JSON/REST	GPL-3.0 / MIT	High	GPL (isolate as service)
App lifecycle	devicectl	Apple + node-devicectl	Adopt	Apple-supported	Apple/Apache-2.0	High	Closed, undoc'd errors
Instrumentation	Frida + Objection	frida/frida	Adopt+wrap	DBI standard	wxWindows/Apache	High	Stock=gadget only; JB fragility
UI MCP	mobile-mcp / iOS- agent-bridge	mobile-next, UgeeCodes	Fork/compose	a11y-first schema	OSS	Medium	Thin; single- device
Scheduler/registry	appium-device-farm (extended)	AppiumTestDistribution	Fork+extend	Parallel session mgmt	MIT+proprietary	Medium	Add capability model
Control plane / MCP gateway	Internal (FastAPI/Go)	—	Build	Nothing covers devices+diag+instr+sched	—	New	Core effort
Model gateway	LiteLLM proxy	BerriAI/litellm	Adopt	Air-gap OpenAI- compatible routing	MIT	High	—
Observability	OTel + Grafana/Tempo/Loki + MinIO	CNCF	Adopt	Standard, on-prem	Apache-2.0	High	—
Deep research (modern iOS)	Corellium + SRD	Corellium / Apple	Adopt (commercial)	Only durable modern-iOS root	Commercial	High	Cost; SRD legal strings

33. Implementation roadmap

- **Phase 0 — Research validation:** Verify experimentally on your actual devices/iOS: (a) tunnel+WDA on each iOS version incl. iOS 18/26; (b) devicectl install/launch/console; (c) pymobiledevice3 logs/crashes/PCAP/DVT; (d) Frida gadget on an org app; (e) palera1n/Dopamine on candidate research devices; (f) signing/provisioning renewal automation. **Gate:** confirm actual capabilities per device class before building.
- **Phase 1 — POC:** 2 stock iPhones + 1 research (jailbroken) iPhone, 1 Mac worker, Claude Code + OpenCode, internal LiteLLM, basic MCP (devices/ui/diagnostics + gated instrumentation), install/launch, logs, screenshots, basic Frida on the research device.
- **Phase 2 — Small Lab (10–20):** scheduler + registry + capability model, health monitoring, parallel execution, CI integration, OTel observability, automated recovery (managed-hub port control).
- **Phase 3 — Production (50–100):** multiple Mac workers, HA control plane, queueing/priorities, worker + research-device isolation (VLANs), device quarantine, centralized artifacts (MinIO), advanced instrumentation, security hardening (Vault, RBAC, audit).
- **Phase 4 — Large (100–500+):** sharded scheduler, fully hands-off health automation, network/artifact capacity planning, Corellium for modern-iOS deep-research scale-out, formal SRE runbooks.

34. Proof-of-concept plan

Deliverables: working `(.mcp.json)` (Claude Code) + `(opencode.json)` (OpenCode) pointing at the HTTP MCP gateway; LiteLLM config with LOW/MID/HIGH aliases; a per-device agent (go-ios tunnel + WDA + pmd3 collectors); one Explorer + one Investigator agent; an evidence bundle to MinIO; a demo goal ("test login on iOS X and Y, investigate any failure, collect logs/screenshots/crashes, classify root cause"). Success metric: fully autonomous, replayable, auditable run across 3 devices with correct capability gating (Frida only on the research device).

Proposed repo layout: `(control-plane/)`, `(scheduler/)`, `(device-registry/)`, `(workers/{ios-stock,ios-research}/)`, `(mcp/{devices,ui,diagnostics,instrumentation}/)`, `(integrations/{appium,xcuitest,coredevice,libimobiledevice,pymobiledevice3,frida,go-ios}/)`, `(agents/{explorer,tester,investigator,security,verifier}/)`, `(ci/)`, `(observability/)`, `(security/)`, `(deployment/{launchd,docker}/)`, `(docs/)`.

35. Production rollout plan

Stage device onboarding (pin iOS, disable OTA via supervision, register UDIDs, pre-provision WDA); establish signing service with auto-renewal; roll out managed hubs with port-control recovery; enable HA control plane + scheduler sharding; segment networks; onboard CI;

institute WDA-restart and re-image runbooks; add Corellium/SRD for modern-iOS research. Define SLOs (device availability, session success rate, recovery MTTR) and alerting.

36. Risks and unresolved limitations

- **iOS version churn:** every major iOS release can break tunnels/WDA (see go-ios iOS 26 issue). Budget continuous maintenance.
- **Jailbreak scarcity:** no durable modern-silicon jailbreak; deep research depends on aging A11–A13 devices, SRD (restricted), or Corellium (cost).
- **Signing/provisioning:** profile expiry and Apple policy shifts are an ongoing burden; air-gap makes PPQ check-in harder (offline profiles mitigate).
- **GPL-3.0 (pymobiledevice3):** isolate as a separate service/subprocess to avoid license entanglement with proprietary code.
- **WDA memory leaks / xcodebuild overhead:** require scheduled restarts and prebuilt WDA.
- **Frida on jailed modern iOS:** fragile (crashes on iOS 17+); gadget scope is app-only.
- **SRD legal constraints:** premises-bound, Apple-owned, report-to-Apple — treat as a special isolated asset.
- **Security:** malicious test apps on research devices — enforce isolation and re-imaging; treat jailbroken devices as hostile.

37. Source / repository bibliography

pymobiledevice3 (github.com/doron88/pymobiledevice3; docs.doron88.github.io/pymobiledevice3; GPL-3.0). go-ios (github.com/danielpaulus/go-ios; MIT). Appium XCUI Test driver (appium.github.io/appium-xcuitest-driver; npm; Apache-2.0). WebDriverAgent via Appium; Thuyen Trinh "WebDriverAgent — The Heart of iOS E2E Testing" (2025). Apple devicectl (developer.apple.com; appium/node-devicectl Apache-2.0; Hex-Rays CoreDevice guide). Frida (frida.re/docs/ios, /docs/gadget; OWASP MASTG; frida issue #3621). libimobiledevice (libimobiledevice.org). Apple SRD (security.apple.com/research-device + security.apple.com/blog; 9to5Mac, AppleInsider, SecurityWeek 2025–2026). Jailbreak (iClarified 2026; cfw.guide; idownloadblog Nullcon 2025 / Lars Fröder; palera1n/Dopamine/TrollStore). Corellium (corellium.com; Cellebrite/GlobeNewswire acquisition PR; 9to5Mac; AppSecSanta 2026). Device farms (DeviceFarmer/stf_ios_support FAQ; appium-device-farm; AWS Device Farm docs; Agoda "200 Mac Minis"). Scaling/USB (Cambrionix SuperSync15 guide + Endpoints Technical Note; Acroname USBHub3c/USBHub3+; Apple Developer Forums cfgutil thread; HowStuffWorks USB). ARTEMIS (github.com/google/artemis). AI/MCP (mobile-next/mobile-mcp; UgeeCodes/iOS-agent-bridge; blitzdotdev/iPhone-mcp; minitap-ai/mobile-use; droidrun/mobilerun; takahiom/arbigent; witchan/ios-mcp; Lakr233/iphone-mcp). Harness/model (code.claude.com/docs/en/mcp; OpenCode MCP config; LiteLLM litellm.ai + github.com/BerriAI/litellm). Logs/forensics (ElcomSoft 2025 unified-log retention; libimobiledevice sysdiagnose).

Recommendations

1. **Adopt the two-tier model now.** Tier 1 (broad): stock + supervised iPhones for UI/app/CI/diagnostics via WDA+pmd3+go-ios+devicectl. Tier 2 (deep): a small, isolated, pinned fleet of A11–A13 jailbroken devices (palera1n rootless / Dopamine) plus, budget permitting, a Corellium appliance and/or an SRD. **Threshold to escalate to Corellium:** when you need root on iOS ≥18 or A14+ silicon, which physical jailbreaks cannot provide.
2. **Build the internal control plane; compose, don't reinvent.** Wrap Appium/WDA, pymobiledevice3, go-ios, devicectl, and Frida behind one HTTP MCP gateway + REST API with dynamic per-device capability negotiation. Isolate pymobiledevice3 (GPL) as a service.
3. **Expose ONE HTTP MCP gateway to both harnesses.** Serves Claude Code (`mcpServers`), HTTP and OpenCode (`mcp`), remote) identically; route models via LiteLLM with LOW/MID/HIGH aliases; default to accessibility-tree observations, screenshots only as fallback.
4. **Prefer deterministic tests; reserve LLM inference for exploration and investigation.** Run XCUI Test/Maestro/Appium scripts for known paths; invoke the AI Explorer/Investigator only for new flows, failures, and cross-device root-cause. Cache AI results on identical UI-tree+goal (arbigent pattern).
5. **Engineer for the USB wall first.** ~10–12 iPhones per USB 2.0 controller with managed Thunderbolt hubs (Acroname/Cambrionix) for per-port power control and endpoint headroom; scale horizontally by adding Mac workers; use detached/prebuilt WDA and scheduled WDA restarts.
6. **Treat jailbroken/malware-test devices as hostile.** Isolated VLAN, no secrets, no control-plane route, re-image after security runs; RBAC-gate shell/Frida/memory tools to research device classes only; immutable audit logs; OTel end-to-end for replayable, auditable runs.
7. **Plan for continuous iOS-version maintenance** as a standing cost, and automate signing/provisioning renewal from day one.

Caveats

- **Vendor scale ratios are unpublished:** BrowserStack/Sauce/HeadSpin/Bitbar do not disclose physical-iPhones-per-Mac numbers; the ~10–15/controller figure derives from USB endpoint limits (Cambrionix) and community cfgutil ceilings, not a single authoritative "devices per Mac" datapoint. Validate on your hardware in Phase 0.
- **Jailbreak landscape is fluid:** tool/device/iOS support changes frequently; the matrix reflects 2025–2026 reporting. Dopamine 3.0 claims are recent — re-verify before procurement.
- **SRD application timing conflict:** press coverage says the 2026 window ended October 31, 2025 while Apple's page currently reads "through October 30, 2026" — confirm directly with Apple before relying on it.
- **Frida-on-stock is app-scoped and fragile on iOS 17+;** do not plan system-wide instrumentation on non-jailbroken devices.
- **Corellium and SRD are not drop-in fleet members** — commercial cost and legal/operational constraints respectively.
- Some capability-matrix cells (e.g. partial DVT process access on stock) depend on exact iOS version and signing state; treat the matrix as a Phase-0 hypothesis to confirm.